

Diagramas UML

BioMove App

Diseño orientado a objetos del sistema BioMove

Versión:	2.0 — Final
Fecha:	13 de junio de 2026
Autor:	Bryan Xavier Calderón Novillo
Institución:	ESPOCH — Ingeniería en Sistemas
Diagramas incluidos:	Clases · Secuencia · Componentes
Herramienta:	Generado con matplotlib Python

1. Diagrama de Clases

El diagrama de clases muestra la arquitectura MVVM de BioMove con cuatro capas bien diferenciadas: View (Flutter Screens), ViewModel (Provider/ChangeNotifier), Repository (abstracción de datos) y Service (Dio, Firebase Auth, WebSocket). La capa de backend incluye los modelos SQLAlchemy y el pipeline de visión artificial.

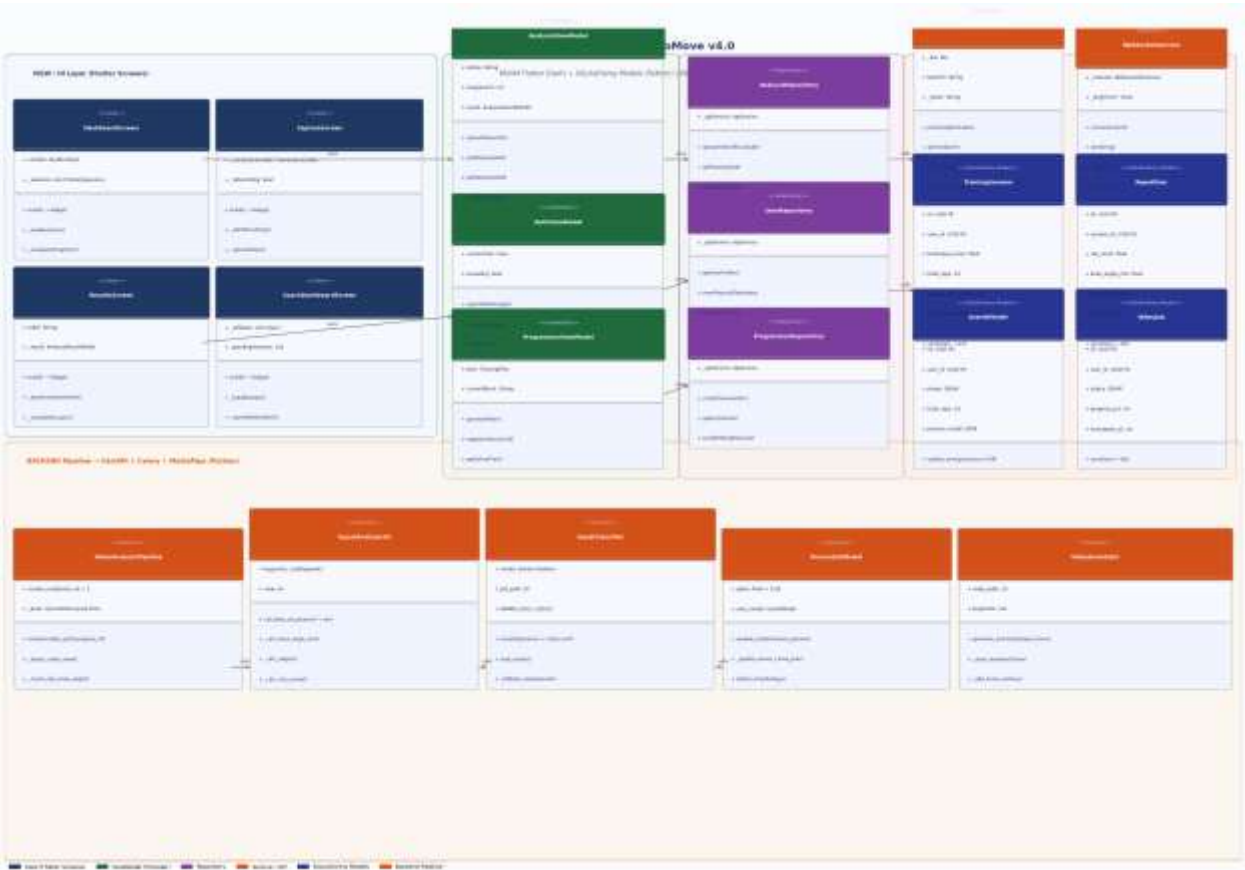


Figura 1. Diagrama de clases de BioMove v4.0 — MVVM Flutter (Dart) + SQLAlchemy Models (Python). Flujo obligatorio: View → ViewModel → Repository → Service → Backend. Nunca View→Repository directamente.

1.1 Descripción de capas

Capa	Clases principales	Responsabilidad
View (Flutter)	DashboardScreen, ResultsScreen, CaptureScreen, CoachDashboardScreen	Presentación UI. Solo usa <code>context.watch<VM>()</code> para leer estado y <code>context.read<VM>()</code> para acciones.
ViewModel (Provider)	AnalysisViewModel, AuthViewModel, ProgressionViewModel	Estado reactivo con <code>ChangeNotifier</code> . Llama a <code>Repositories</code> . Llama <code>notifyListeners()</code> tras cambios.
Repository	AnalysisRepository, UserRepository, ProgressionRepository	Abstracción de fuente de datos. Puede cambiar <code>API→cache</code> sin afectar <code>ViewModel</code> .
Service	ApiService (Dio+JWT), WebSocketService, VideoAnnotator	Comunicación con backend, <code>WebSocket</code> y herramientas especializadas.
SQLAlchemy Models	User, VideoJob, TrainingSession, Repetition, UserAIModel	ORM Python. 12 entidades en <code>SQLite/PostgreSQL</code> .

Capa	Clases principales	Responsabilidad
Pipeline Backend	VideoAnalysisPipeline, SquatAnalyzer40, SquatClassifier, PersonalAIModel	Procesamiento Celery: MediaPipe → 40 params → sklearn → EMA.

2. Diagrama de Secuencia — Flujo de Análisis de Video

El diagrama de secuencia muestra el flujo completo del caso de uso CU-002 y CU-003: desde que el usuario sube el video hasta que recibe los resultados. Involucra 8 participantes: Usuario, CaptureScreen, AnalysisViewModel, ApiService, FastAPI, Celery Worker, Pipeline MediaPipe y SQLite.

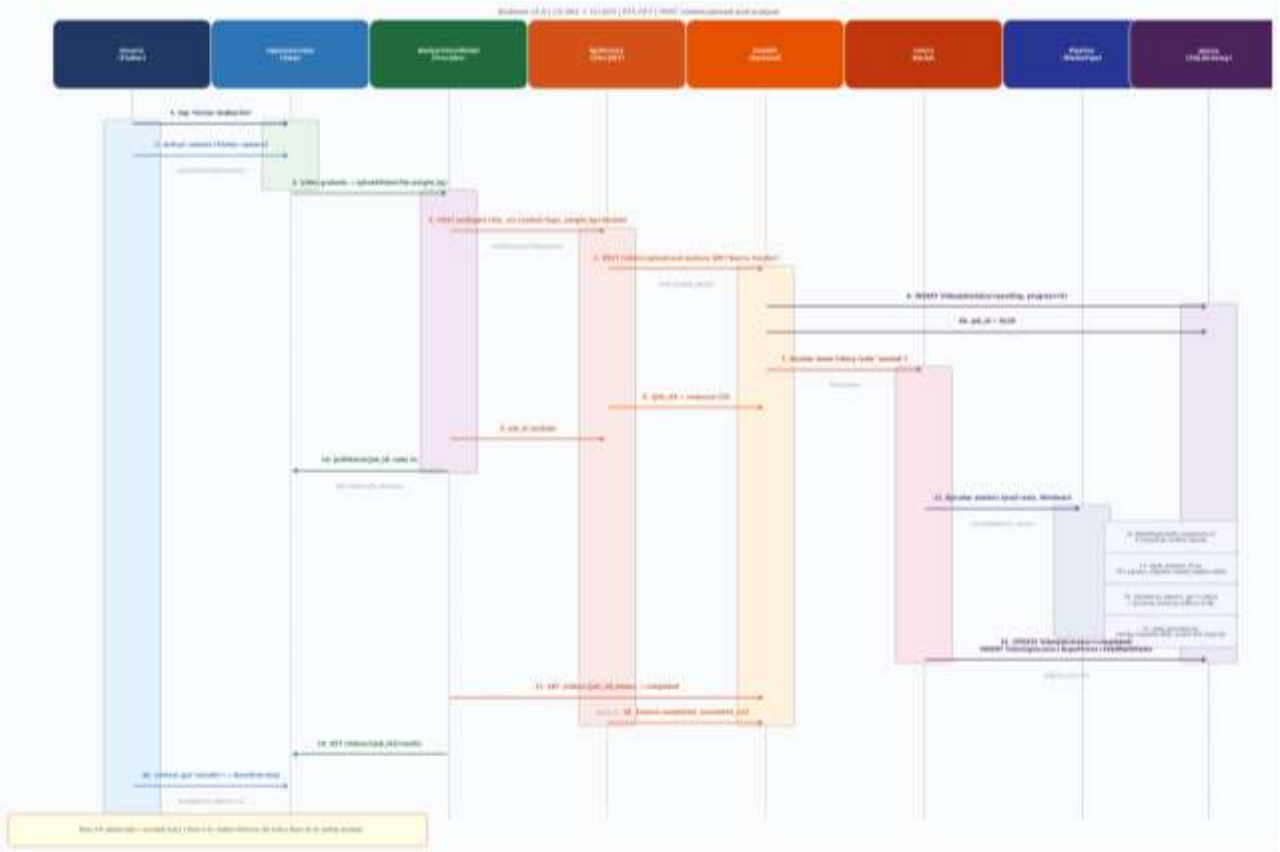


Figura 2. Diagrama de secuencia — Flujo upload + análisis asíncrono (CU-002 + CU-003). 20 pasos desde la grabación hasta la presentación de resultados. El análisis es asíncrono (Celery) con polling cada 3s desde Flutter.

2.1 Fases del flujo de análisis

Pasos	Fase	Descripción técnica
1-8	Upload y encolado	Flutter graba/selecciona video → Dio multipart POST → FastAPI crea VideoJob → Celery encola en cola "normal" → job_id retornado al cliente
9-15	Análisis asíncrono Celery	Worker ejecuta: MediaPipe (33 kpts, frames impares) → máquina estados ángulo rodilla → squat_analyzer_40.py (40+ params) → sklearn 5 clases → EMA $\alpha=0.08$ → video_annotator.py
16	Almacenamiento SQLite	Celery guarda TrainingSession + Repetitions + FeedbackItems + actualiza VideoJob.status="completed"

Pasos	Fase	Descripción técnica
17-20	Polling y presentación	Flutter detecta status=completed → GET /results → context.go("/results") → ResultsScreen muestra ScoreGauge + video anotado